Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 05

Desarrollo de aplicaciones Web: Validación y Manejo de Errores.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: VALIDACIÓN Y MANEJO DE ERRORES

I. Capacidades

Implementar validación y manejo de excepciones en APIs.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

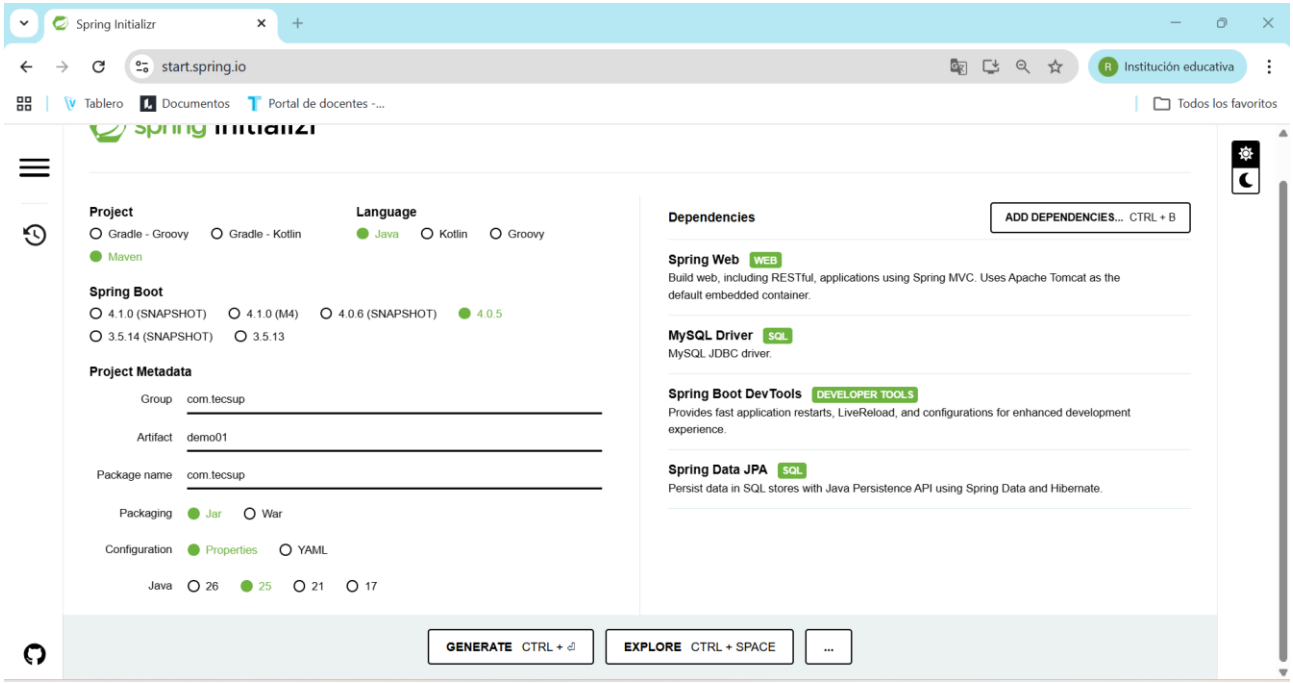
VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

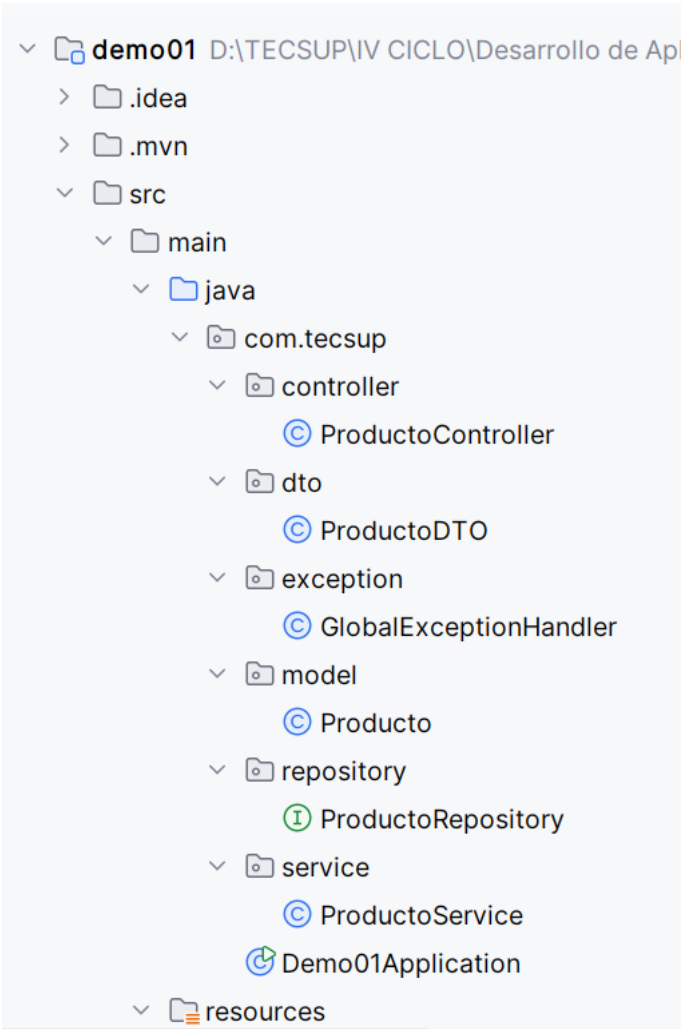
VII. Procedimiento

Laboratorio 01

1.- Crear proyecto en Spring Initializr:



2.- Estructura:



3.- Crear la base de datos con MySql.

```
CREATE DATABASE laboratorio_api;
USE laboratorio_api;
```

4.- Clase Producto

```
package com.tecsup.model;


import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private double precio;
    private int stock;

    // Getters y Setters


    public Producto() {
    }

    public Producto(Long id, String nombre, double precio, int stock)
{
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
```

```
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}
```

5.- Ejecute la aplicación y revisar la base de datos.

6.- Interface ProductoRepository

```
package com.tecsup.repository;

import com.tecsup.model.Producto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductoRepository extends JpaRepository<Producto,
Long> {
}
```

7.- Clase ProductoService

```
package com.tecsup.service;

import com.tecsup.model.Producto;
import com.tecsup.repository.ProductoRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ProductoService {

    @Autowired
    private ProductoRepository repo;

    public List<Producto> listar() {
        return repo.findAll();
```

```
    }

    public Producto guardar(Producto p) {
        return repo.save(p);
    }

    public Producto obtener(Long id) {
        return repo.findById(id).orElse(null);
    }

    public void eliminar(Long id) {
        repo.deleteById(id);
    }

}
```

8.- Clase ProductoDTO.

```
package com.tecsup.dto;

import jakarta.validation.constraints.*;

public class ProductoDTO {

    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;

    @Positive(message = "El precio debe ser mayor a 0")
    private double precio;

    @Min(value = 0, message = "El stock no puede ser negativo")
    private int stock;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
```

```
        this.stock = stock;
    }
}
```

9.- Clase GlobalExceptionHandler

```
package com.tecsup.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
manejarErrores(MethodArgumentNotValidException ex) {

        Map<String, String> errores = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });

        return ResponseEntity.badRequest().body(errores);
    }
}
```

10.- Clase ProductoController

```
package com.tecsup.controller;

import java.util.List;

import com.tecsup.dto.ProductoDTO;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.tecsup.model.Producto;
import com.tecsup.service.ProductoService;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    @Autowired
```

```
    private ProductoService service;

    // GET
    @GetMapping
    public ResponseEntity<List<Producto>> listar() {
        return ResponseEntity.ok(service.listar());
    }

    // POST (con DTO + validación)
    @PostMapping
    public ResponseEntity<?> guardar(@Valid @RequestBody ProductoDTO
dto) {

        Producto p = new Producto();
        p.setNombre(dto.getNombre());
        p.setPrecio(dto.getPrecio());
        p.setStock(dto.getStock());

        Producto guardado = service.guardar(p);

        return ResponseEntity.status(201).body(guardado);
    }

    // GET por ID
    @GetMapping("/{id}")
    public ResponseEntity<?> obtener(@PathVariable Long id) {

        Producto p = service.obtener(id);

        if (p == null) {
            return ResponseEntity.status(404).body("Producto no
encontrado");
        }

        return ResponseEntity.ok(p);
    }

    // PUT
    @PutMapping("/{id}")
    public ResponseEntity<?> actualizar(@PathVariable Long id,
                                        @Valid @RequestBody
ProductoDTO dto) {

        Producto existente = service.obtener(id);

        if (existente == null) {
            return ResponseEntity.status(404).body("Producto no
existe");
        }

        existente.setNombre(dto.getNombre());
        existente.setPrecio(dto.getPrecio());
        existente.setStock(dto.getStock());
```

```
        return ResponseEntity.ok(service.guardar(existente));
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<?> eliminar(@PathVariable Long id) {

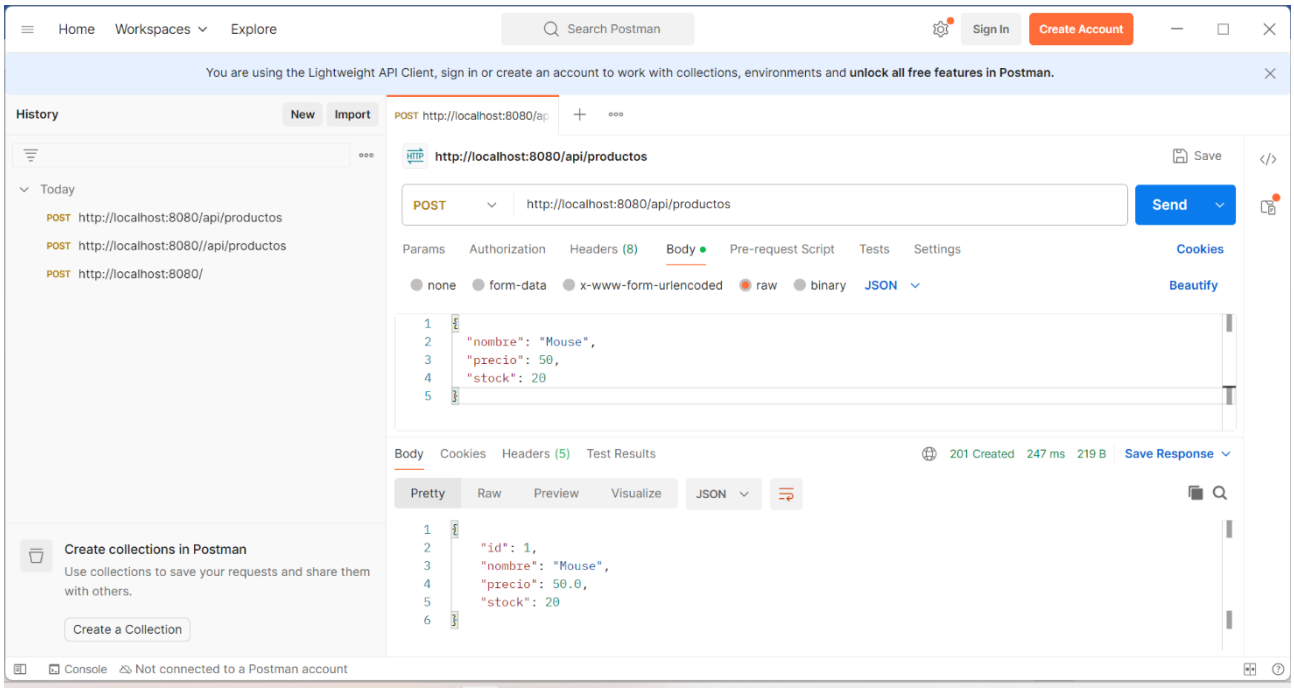
        Producto p = service.obtener(id);

        if (p == null) {
            return ResponseEntity.status(404).body("No existe");
        }

        service.eliminar(id);

        return ResponseEntity.ok("Eliminado correctamente");
    }
}
```

9.- Ejecutar la aplicación y realizar pruebas con Postman



POST: http://localhost:8080/api/productos

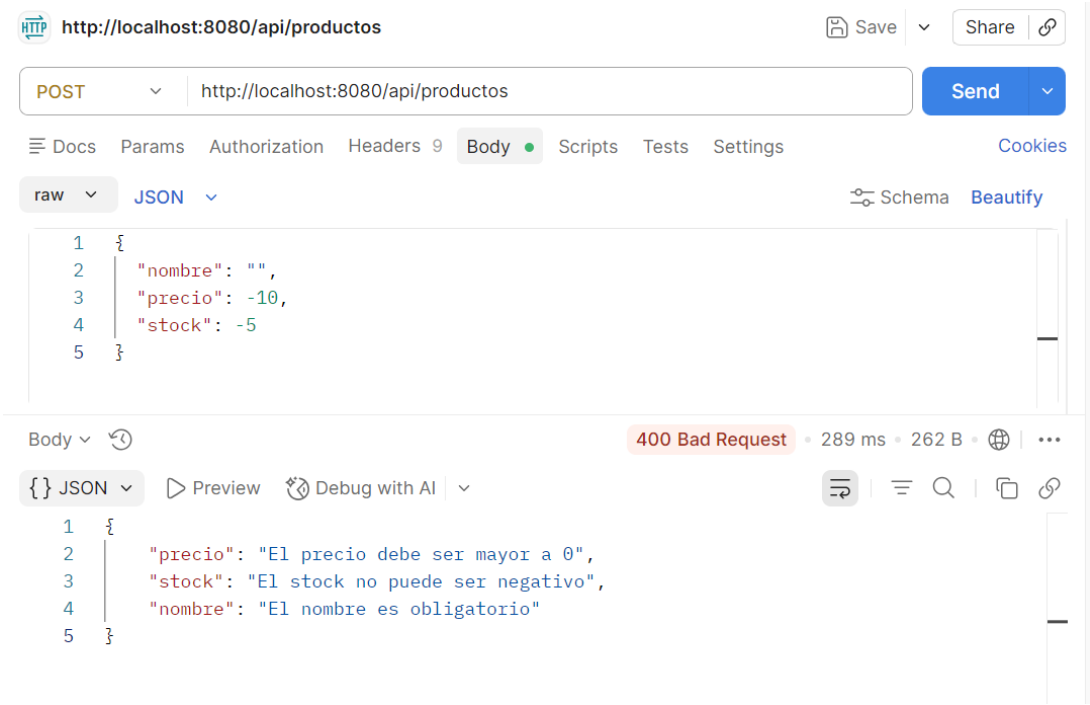
```
{
  "nombre": "Monitor",
  "precio": 50,
  "stock": 20
}
```

GET: http://localhost:8080/api/productos

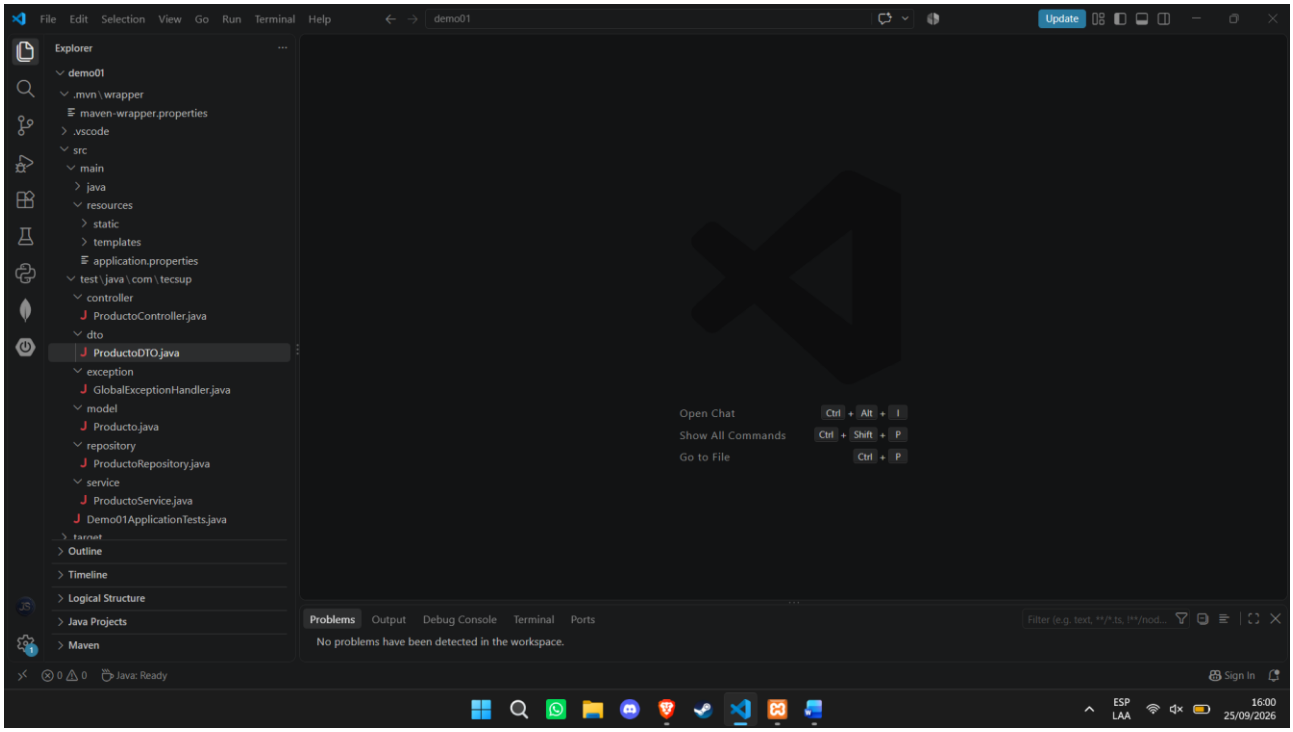
PUT: http://localhost:8080/api/productos/1

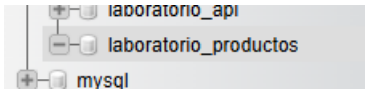
```
{
  "nombre": "Monitor actualizado",
  "precio": 50,
  "stock": 20
}
```

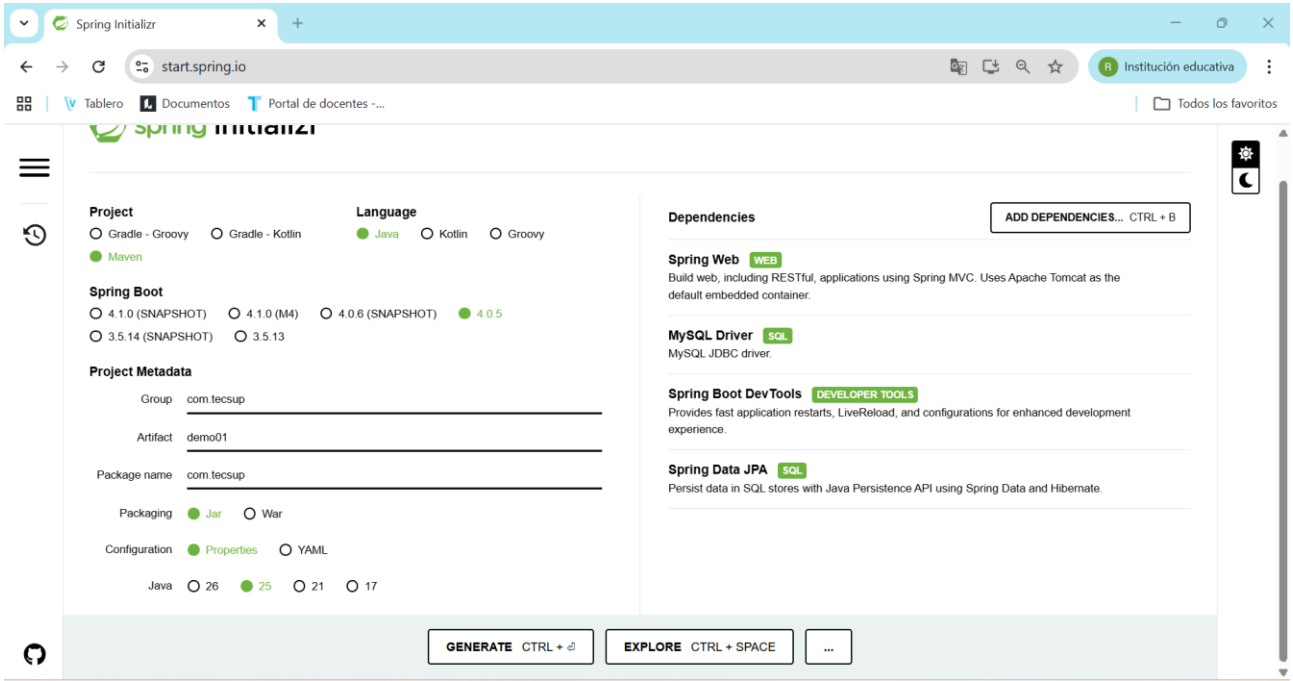
DELETE: http://localhost:8080/api/productos/1



EVIDENCIA:


File Edit Selection View Go Run Terminal Help
demo01
Update
Explorer
demo01
.mvn\wrapper
maven-wrapper.properties
.vscode
src
main
java
resources
static
templates
application.properties
test\java\com\tecsup
controller
ProductoController.java
dto
ProductoDTO.java
exception
GlobalExceptionHandler.java
model
Producto.java
repository
ProductoRepository.java
service
ProductoService.java
Demo01ApplicationTests.java
target
Outline
Timeline
Logical Structure
Java Projects
Maven
Open Chat Ctrl + Alt + I
Show All Commands Ctrl + Shift + P
Go to File Ctrl + P
Problems
Output
Debug Console
Terminal
Ports
No problems have been detected in the workspace.
Java: Ready
Sign In
ESP LAA
16:00
25/09/2026


laboratorio_api
laboratorio_productos
mysql


Spring Initializr
start.spring.io
Institución educativa
Tablero
Documentos
Portal de docentes -...
Todos los favoritos
Project
Gradle - Groovy
Gradle - Kotlin
Maven
Language
Java
Kotlin
Groovy
Spring Boot
4.1.0 (SNAPSHOT)
4.1.0 (M4)
4.0.6 (SNAPSHOT)
4.0.5
3.5.14 (SNAPSHOT)
3.5.13
Project Metadata
Group com.tecsup
Artifact demo01
Package name com.tecsup
Packaging Jar War
Configuration Properties YAML
Java 26 25 21 17
Dependencies
ADD DEPENDENCIES... CTRL + B
Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
MySQL Driver SQL
MySQL JDBC driver.
Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.
Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
GENERATE CTRL + ⏎
EXPLORE CTRL + SPACE
...

Actividad

A partir del laboratorio desarrollado, implementar una nueva funcionalidad en el API de productos.

Agregar un nuevo campo:

private String categoria;

Implementar un endpoint que permita buscar productos por nombre.

EVIDENCIA: CAMBIOS

Producto.java

Primero, armé la clase Producto y le puse la anotación @Entity para que represente mi tabla en la base de datos. Le definí los campos básicos (id, nombre, precio, stock) y, gracias a la configuración de Spring Boot, la tabla se creó solita en MySQL sin que yo tuviera que escribir el código SQL a mano.

```
package com.tecsup.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private double precio;
    private int stock;

    // 1. Agregamos el nuevo campo según la actividad
    private String categoria;

    public Producto() {}

    public Producto(Long id, String nombre, double precio, int stock, String categoria) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
        this.categoria = categoria;
    }

    // Getters y Setters existentes...
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
```

```
    public void setNombre(String nombre) { this.nombre = nombre; }
    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }
    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }

    // 2. Agregamos el Getter y Setter de categoría
    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria = categoria; }
}
```

ProductoDTO.java

Después creé el ProductoDTO por un tema de seguridad, para no recibir los datos directamente en el modelo de la base de datos. En esta clase aproveché de meterle las validaciones clave (como @NotBlank y @Positive) para asegurarme de que no se puedan guardar productos con nombres en blanco o con precios y stock negativos.

```
// ... debajo de los atributos que ya tienes (nombre, precio, stock)

    @NotBlank(message = "La categoría es obligatoria")
    private String categoria;

    // Agrega su Getter y Setter al final de la clase
    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria = categoria; }
```

ProductoRepository.java

En la capa de datos, simplemente creé esta interfaz haciéndola heredar de JpaRepository. Literalmente esto me ahorró tener que escribir todo el código SQL, ya que Spring Data se encarga por detrás de darme las funciones para guardar, actualizar, buscar y borrar productos.

```
package com.tecsup.repository;

import com.tecsup.model.Producto;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface ProductoRepository extends JpaRepository<Producto, Long> {

    // Método automático para buscar productos por nombre (ignorando mayúsculas/minúsculas)
    List<Producto> findByNombreContainingIgnoreCase(String nombre);
}
```

ProductoService.java

Luego armé el ProductoService para centralizar la lógica de mi aplicación. Acá inyecté el repositorio y armé los métodos del CRUD. Hacer esto me permitió mantener la arquitectura limpia, separando completamente las reglas del negocio de lo que es la recepción de peticiones web.

```
// ... debajo de los métodos que ya tienes (listar, guardar, etc.)
```

```
  public List<Producto> buscarPorNombre(String nombre) {
    return repo.findByNombreContainingIgnoreCase(nombre);
  }
```

ProductoController.java

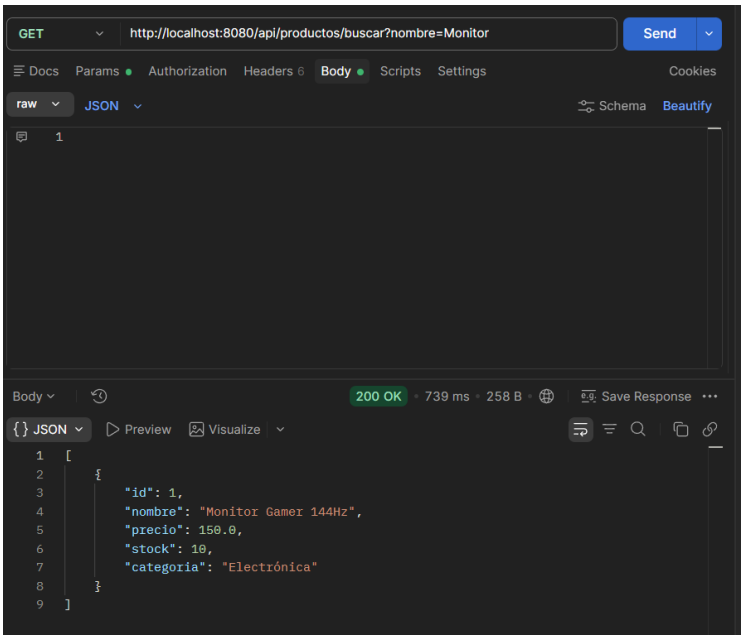
Por último, monté el ProductoController para exponer mi API en la ruta /api/productos. Conecté los métodos GET, POST, PUT y DELETE con mi servicio. El paso más importante acá fue agregarle la anotación @Valid a los datos que entran en el POST y PUT, para que las restricciones del DTO realmente se activen cuando hago mis pruebas desde Postman.

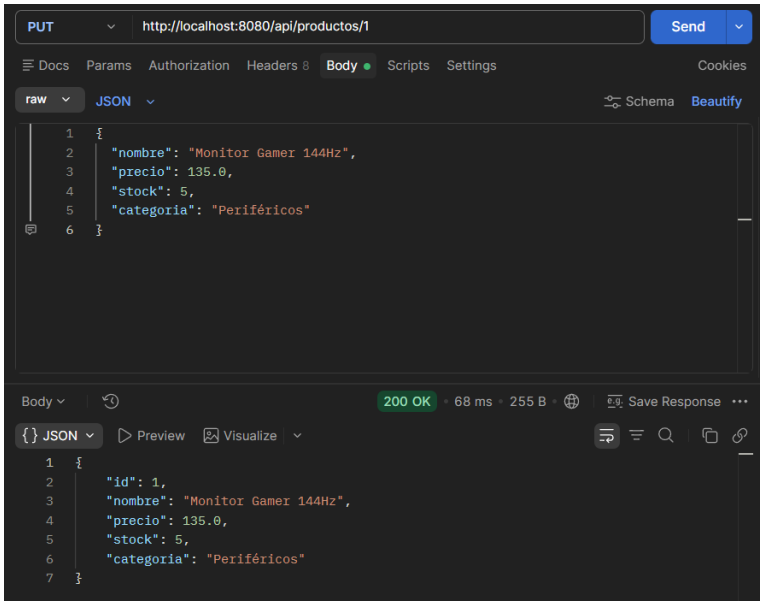
```
// Modifica tu método guardar para incluir la categoría:
  @PostMapping
  public ResponseEntity<?> guardar(@Valid @RequestBody ProductoDTO dto) {
    Producto p = new Producto();
    p.setNombre(dto.getNombre());
    p.setPrecio(dto.getPrecio());
    p.setStock(dto.getStock());
    p.setCategoria(dto.getCategoria()); // <--- Nueva línea
    Producto guardado = service.guardar(p);
    return ResponseEntity.status(201).body(guardado);
  }

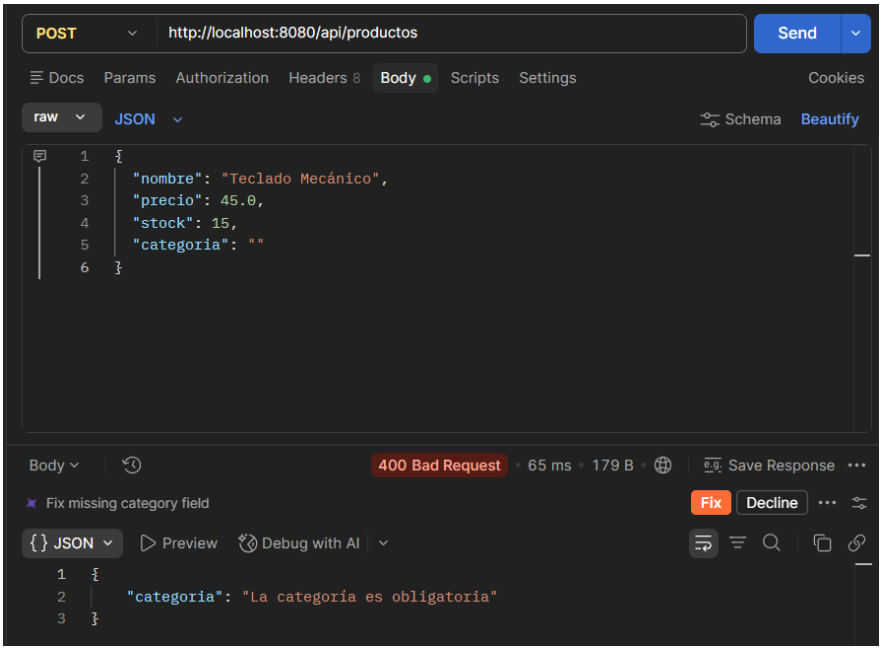
  // Modifica tu método actualizar para incluir la categoría:
  @PutMapping("/{id}")
  public ResponseEntity<?> actualizar(@PathVariable Long id, @Valid @RequestBody ProductoDTO dto)
{
    Producto existente = service.obtener(id);
    if (existente == null) {
      return ResponseEntity.status(404).body("Producto no existe");
    }
    existente.setNombre(dto.getNombre());
    existente.setPrecio(dto.getPrecio());
    existente.setStock(dto.getStock());
    existente.setCategoria(dto.getCategoria()); // <--- Nueva línea
    return ResponseEntity.ok(service.guardar(existente));
  }

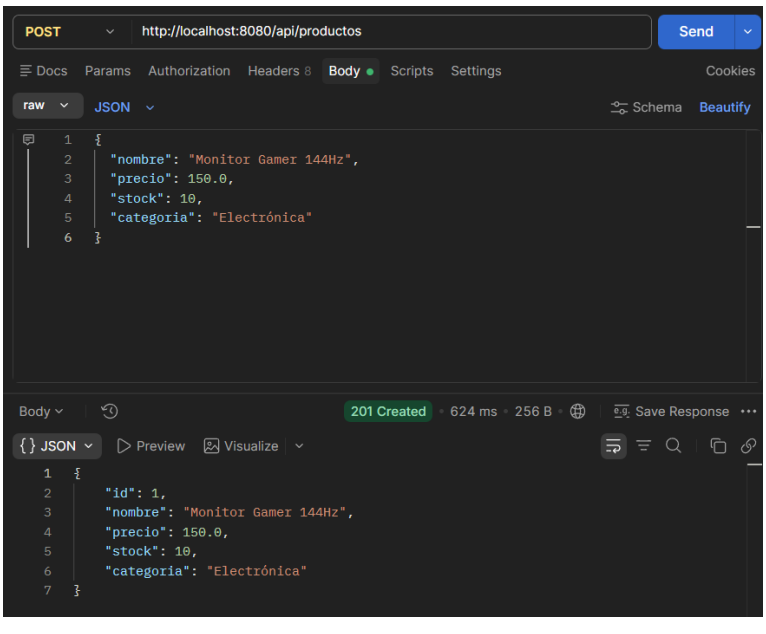
  // AGREGA EL NUEVO ENDPOINT DE BÚSQUEDA AL FINAL DE LA CLASE:
  @GetMapping("/buscar")
  public ResponseEntity<List<Producto>> buscarPorNombre(@RequestParam String nombre) {
    List<Producto> productos = service.buscarPorNombre(nombre);
    if (productos.isEmpty()) {
      return ResponseEntity.status(404).body(null);
    }
    return ResponseEntity.ok(productos);
  }
```

PRUEBAS EN POSTMAN









Responder las siguientes preguntas:

1. ¿Por qué usamos un DTO en lugar de exponer directamente la entidad?

Usamos un DTO (Data Transfer Object) para separar la capa de presentación de la base de datos. Esto evita exponer datos sensibles de la entidad, previene vulnerabilidades de asignación masiva (Mass Assignment) y nos permite moldear y validar únicamente la información exacta que necesitamos recibir del cliente, sin importar cómo esté construida la tabla internamente en MySQL.

2. ¿Qué función cumple @Valid?

La anotación @Valid se encarga de disparar el proceso de validación antes de que los datos ingresen a la lógica del método. Al colocarla junto al @RequestBody, obliga a Spring a verificar que los datos enviados cumplan con las reglas definidas en el DTO (como @NotBlank o @Positive). Si no las cumplen, rechaza la petición automáticamente lanzando una excepción.

3. ¿Qué pasaría si eliminamos @RestControllerAdvice?

Se perdería el manejo centralizado y controlado de las excepciones. En lugar de devolver un JSON limpio y amigable que indique específicamente qué campos fallaron (ej. "El precio debe ser mayor a 0"), el sistema devolvería la página de error genérica de Spring Boot (WhiteLabel Error Page) o un extenso y confuso "stack trace" en formato 500 Internal Server Error o 400 Bad Request, lo cual es una mala práctica para el cliente que consume la API.

4. ¿Qué hace @Autowired?

Implementa el patrón de Inyección de Dependencias de Spring Boot. Permite que el contenedor de Spring instancie e inyecte automáticamente los objetos necesarios en memoria (como inyectar el ProductoRepository dentro del ProductoService) sin que el desarrollador tenga que instanciarlos manualmente usando la palabra clave new.

5. ¿Cuál es la diferencia entre:
   - save(): Es un método del repositorio que guarda un objeto nuevo en la base de datos (INSERT) o actualiza uno existente si el ID provisto ya existe (UPDATE
   - findById(): Realiza una consulta (SELECT) para buscar un registro específico en la tabla utilizando su clave primaria (ID). Devuelve un contenedor Optional para manejar de forma segura el caso en que no se encuentre el registro.
   - findAll(): Realiza una consulta general (SELECT * FROM) para recuperar absolutamente todos los registros existentes en la tabla y los devuelve estructurados dentro de una colección (Lista).

OBSERVACIONES: *(Las observaciones son las notas aclaratorias y problemas que se pudo presentar en el desarrollo del laboratorio)*

- Al configurar la conexión en el archivo application.properties, es un requisito estricto mantener encendido el servicio de MySQL a través del panel de XAMPP; de lo contrario, la aplicación genera un error de "Connection Refused" y detiene su ejecución.
- Durante la creación del proyecto, fue necesario agregar manualmente la dependencia de Spring Boot Starter Validation en el archivo pom.xml, ya que sin esta librería las anotaciones clave del laboratorio (@NotBlank, @Positive, @Min) no eran reconocidas por el entorno de desarrollo.
- Se observó que el uso de la propiedad spring.jpa.hibernate.ddl-auto=update resulta de gran utilidad en etapa de desarrollo, ya que mapea y genera la tabla de base de datos automáticamente sin necesidad de escribir scripts SQL de forma manual.
- Para evitar conflictos de integridad relacional o mezcla de tablas con proyectos de laboratorios pasados, fue una excelente práctica generar una base de datos limpia e independiente (laboratorio_productos) exclusivamente para este requerimiento.
- Durante las pruebas con Postman, se verificó que el cuerpo (Body) de las peticiones POST y PUT debe enviarse estrictamente configurado en formato raw estructurado como JSON. Cualquier otro formato genera errores HTTP 415 (Unsupported Media Type).

CONCLUSIONES: *(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además de aportar una opinión critica de lo realizado)*

Indicar 5 conclusiones:

- La implementación de anotaciones de validación (como @Positive y @Min) garantiza la integridad de los datos desde su punto de entrada. Esto asegura que la base de datos jamás reciba registros corruptos o lógicamente imposibles, como precios o stocks en valores negativos.
- Separar el sistema empleando una arquitectura multicapa (Entity, Repository, Service, Controller) establece un flujo de datos ordenado. Esta práctica permite que el código sea altamente mantenible, ya que cada clase asume una responsabilidad única y especializada.
- El uso estratégico de la anotación @RestControllerAdvice demuestra ser un pilar fundamental en el desarrollo de APIs profesionales, al permitir interceptar errores globales y transformarlos en respuestas HTTP estandarizadas y comprensibles para el frontend o el usuario final.
- Se comprobó de manera práctica la eficacia del patrón DTO, confirmando que funciona como una barrera de seguridad necesaria para filtrar la comunicación externa, impidiendo exponer el diseño interno de la base de datos ante el exterior.
- Las pruebas simuladas en herramientas de testeo como Postman son indispensables para garantizar el correcto comportamiento del sistema. Comprobar escenarios de éxito (200 OK / 201 Created) y forzar escenarios de error (400 Bad Request) asegura la resiliencia del software antes de su paso a producción.